

Rapport de projet informatique

Détection automatique des Fake News

Projet réalisé du 1 Avril 2025 au 23 Mai 2025

Lien GitHub :

<https://github.com/Galaxou/fake-news-detector>

Membres du groupe

OUATIK Marwane 45018118
GHANEM Rayan 45017436
BENDJELLOUL Rayane 4501665
BENELKHAZNADJI Nadir 4501665

Table des matières

1	Introduction	3
2	Environnement de travail	3
3	Description du projet et objectifs	3
3.1	Mise en contexte	3
3.2	Objectifs	3
4	Bibliothèques, Outils et technologies	4
5	Travail réalisé	4
6	Difficultés rencontrées	5
7	Bilan	6
7.1	Conclusion	6
7.2	Perspectives	6
8	Webographie	6
9	Annexes	7
A	Cahier des charges	7
B	Exemple d'exécution du projet	7
C	Manuel utilisateur	8

1 Introduction

Dans un monde où l'information circule à une vitesse sans précédent, la désinformation constitue un danger croissant pour les sociétés démocratiques. Les *fake news* — ou fausses informations — se propagent via les réseaux sociaux, les médias en ligne et les messageries instantanées, rendant difficile pour le grand public de distinguer le vrai du faux.

Dans ce projet, nous avons conçu un système automatique de détection des fake news s'appuyant sur des techniques d'apprentissage automatique (*Machine Learning*). Pour ce faire, nous utilisons deux jeux de données : **Fake.csv** et **True.csv**, contenant respectivement des articles faux et des articles authentiques [DATASET].

Ce rapport décrit les choix techniques effectués, les outils utilisés, les difficultés rencontrées et les résultats obtenus.

2 Environnement de travail

Nous avons travaillé en **Python**, langage incontournable en Data Science grâce à sa syntaxe claire et sa richesse en bibliothèques spécialisées.

Concernant les environnements de développement, nous avons utilisé deux outils complémentaires :

- **JupyterLite (WASM)** : environnement principal d'exécution, accessible directement via un navigateur web sans installation locale. Les bibliothèques s'installent via la commande `!mamba install`.
- **Visual Studio Code** : utilisé pour l'édition locale du code source, grâce à sa légèreté et ses nombreuses extensions Python.

Pour la rédaction du rapport, nous avons utilisé **Overleaf**, une plateforme en ligne de rédaction $\text{\LaTeX}$ permettant la collaboration en temps réel [OVERLEAF].

3 Description du projet et objectifs

3.1 Mise en contexte

Le projet s'appuie sur deux fichiers CSV issus d'un dataset public largement utilisé dans la recherche sur la désinformation. **Fake.csv** contient des articles de presse identifiés comme faux, tandis que **True.csv** contient des articles authentiques. Après fusion et mélange aléatoire, le dataset comprend plusieurs dizaines de milliers d'articles étiquetés.

L'idée centrale est d'entraîner un modèle de classification supervisée capable, à partir du texte brut d'un article, de prédire s'il s'agit d'une fake news ou d'une information authentique.

3.2 Objectifs

Les objectifs de notre projet sont :

- **Traitement du langage naturel (NLP)** : nettoyer et préparer les textes (suppression des stopwords, URLs, caractères spéciaux).

- **Classification binaire** : distinguer les fausses informations des vraies avec une précision maximale.
- **Détection mathématique** : identifier et vérifier automatiquement les expressions mathématiques saisies par l'utilisateur.
- **Interface utilisateur** : permettre à un utilisateur de soumettre un texte et d'obtenir un verdict **FAKE NEWS** ou **AUTHENTIC NEWS**.
- **Sauvegarde du modèle** : exporter le modèle entraîné pour une réutilisation sans ré-entraînement.

4 Bibliothèques, Outils et technologies

Bibliothèques Python utilisées

- **pandas** : chargement et manipulation des datasets CSV.
- **numpy** : opérations numériques de base.
- **scikit-learn** : vectorisation TF-IDF (**TfidfVectorizer**), modèle **LogisticRegression**, métriques d'évaluation (**accuracy_score**, **classification_report**, **confusion_matrix**).
- **nltk** : traitement du langage naturel, suppression des stopwords anglais.
- **joblib** : sérialisation et sauvegarde du modèle et du vectoriseur entraînés.
- **ast / re / operator** : analyse syntaxique et évaluation sécurisée des expressions mathématiques (sans **eval()**).

Technologies clés

- **TF-IDF** (*Term Frequency-Inverse Document Frequency*) : méthode de vectorisation qui transforme les textes en vecteurs numériques en pondérant l'importance relative de chaque mot.
- **Régression Logistique** : algorithme de classification binaire supervisée, simple, interprétable et très performant sur des représentations TF-IDF.

5 Travail réalisé

Répartition des tâches

Nous avons divisé le travail en trois parties afin que chacun puisse contribuer pleinement au projet :

- **Partie 1 — OUATIK Marwane** : Préparation des données, chargement des CSV, nettoyage des textes (fonction **clean_text**), étiquetage et fusion des datasets.
- **Partie 2 — GHANEM Rayan** : Vectorisation TF-IDF, entraînement du modèle de régression logistique, évaluation (**accuracy**, matrice de confusion, rapport de classification), sauvegarde du modèle avec **joblib**.
- **Partie 3 — BENDJELLOUL Rayane et BENELKHAZNAJDI Nadir** : Détection et évaluation sécurisée des expressions mathématiques via le module **ast**, boucle interactive utilisateur.

Fonctionnalités prévues et réalisées

Fonctionnalité	Réalisée ?	Remarques
Chargement et fusion des datasets	Oui	—
Nettoyage NLP des textes	Oui	—
Vectorisation TF-IDF	Oui	—
Entraînement régression logistique	Oui	Précision > 98 %
Sauvegarde du modèle (.pkl)	Oui	—
Détection mathématique sécurisée	Oui	Sans <code>eval()</code>
Interface utilisateur interactive	Oui	Boucle <code>while</code>
Interface web (Flask/Streamlit)	Non	Manque de temps

TABLE 1 – Liste des fonctionnalités prévues et leur état de réalisation

Le tableau 1 résume l'ensemble des fonctionnalités. L'interface web n'a pas pu être réalisée par manque de temps lors de la phase finale du projet.

6 Difficultés rencontrées

- **Compréhension des données** : Il a fallu analyser le contenu des fichiers CSV et s'assurer que les labels étaient correctement attribués avant tout entraînement.
- **Environnement JupyterLite (WASM)** : L'installation des bibliothèques nécessite `mamba` et non `pip`, ce qui est inhabituel. La gestion des chemins de fichiers dans un environnement navigateur a également posé des problèmes.
- **Sécurité de l'évaluateur mathématique** : L'utilisation de `eval()` étant dangereuse, nous avons dû implémenter un évaluateur basé sur le module `ast`, qui analyse l'arbre syntaxique abstrait de l'expression.
- **Limites des LLM** : Nous avons utilisé ChatGPT et Mistral comme outils d'assistance. Si utiles pour déboguer, ils ont parfois produit du code qui semblait correct mais ne fonctionnait pas dans notre environnement spécifique.

7 Bilan

7.1 Conclusion

Ce projet nous a permis de mettre en pratique les grandes étapes d'un pipeline de Machine Learning appliqué au traitement du langage naturel : préparation des données, extraction de features, entraînement, évaluation et déploiement sous forme d'interface interactive. Le modèle atteint une précision supérieure à 98 % sur les données de test, ce qui confirme l'efficacité de l'approche TF-IDF + régression logistique pour ce type de tâche.

7.2 Perspectives

Plusieurs améliorations pourraient être apportées à notre système :

- Tester des modèles plus sophistiqués : transformers (BERT, RoBERTa) pour une meilleure compréhension sémantique.
- Analyser également le titre et la source de l'article.
- Créer une interface web avec Flask ou Streamlit.
- Étendre la détection à d'autres langues, notamment le français.

8 Webographie

Références

- [SKL-DOC] <https://scikit-learn.org/stable/>
[NLTK] <https://www.nltk.org/>
[DATASET] <https://www.kaggle.com/clmentbisailon/fake-and-real-news-dataset>
[OVERLEAF] <https://www.overleaf.com/>
[GPT] <https://chat.openai.com/>
[MISTRAL] <https://chat.mistral.ai/>

9 Annexes

Annexe A : Cahier des charges

1. Introduction

Le projet vise à développer un programme de détection automatique des fake news à partir de textes en anglais, en s'appuyant sur le NLP et le Machine Learning.

2. Objectifs

- Développer un pipeline complet de traitement et classification de textes.
- Offrir une interface interactive pour soumettre un article.
- Gérer la vérification des expressions mathématiques.
- Sauvegarder le modèle entraîné pour réutilisation.

3. Fonctionnalités

- Nettoyage NLP : stopwords, URLs, caractères spéciaux.
- Vectorisation TF-IDF.
- Classification binaire : FAKE / AUTHENTIC.
- Évaluation mathématique sécurisée (sans `eval()`).

4. Planning : du 1 Avril au 23 Mai 2025

- Phase 1 : Analyse des besoins (1 semaine).
- Phase 2 : Préparation des données et pipeline NLP (2 semaines).
- Phase 3 : Entraînement et évaluation du modèle (1 semaine).
- Phase 4 : Interface utilisateur et tests finaux (1 semaine).

Annexe B : Exemple d'exécution du projet

Pour cet exemple, nous avons testé les entrées suivantes dans la boucle interactive :

Sortie console

```
1 =====
2 DETECTION AUTOMATIQUE DES FAKE NEWS GRACE A L'IA
3 =====
4
5 Entree : 2 + 2 = 4
6 Resultat : AUTHENTIC NEWS
7
8 Entree : 10 > 5
9 Resultat : AUTHENTIC NEWS
10
11 Entree : 5 * 7 = 20
12 Resultat : FAKE NEWS
13
14 Entree : NASA discovered water on Mars
15 Resultat : AUTHENTIC NEWS
16
17 Entrez une information (ou tapez exit) : exit
18 Programme arrete
```

Les trois premiers exemples illustrent la détection mathématique : $2 + 2 = 4$ est vrai (AUTHENTIC), $10 > 5$ est vrai (AUTHENTIC), mais $5 \times 7 = 20$ est faux (FAKE NEWS). Le dernier exemple illustre la classification NLP sur un texte journalistique.

Annexe C : Manuel utilisateur

Installation des dépendances

Dans un environnement JupyterLite :

```
!mamba install pandas numpy scikit-learn nltk joblib
```

Dans un environnement Python standard :

```
pip install pandas numpy scikit-learn nltk joblib
```

Fichiers nécessaires

- `Fake.csv` — dataset des articles faux
- `True.csv` — dataset des articles authentiques
- Le notebook ou script Python principal

Structure du dossier recommandée

```
projet_fakenews/  
|-- fake_news_detection.ipynb  
|-- Fake.csv  
|-- True.csv  
|-- fake_news_model.pkl          (genere apres entrainement)  
|-- tfidf_vectorizer.pkl        (genere apres entrainement)
```

Exécution

1. Placer `Fake.csv` et `True.csv` dans le même dossier que le script.
2. Exécuter toutes les cellules dans l'ordre.
3. Saisir un texte ou une expression mathématique à l'invite.
4. Taper `exit` pour quitter le programme.